

SZYMON ŻYREK

SOFTWARE / SYSTEMS ARCHITECT · SENIOR SOFTWARE ENGINEER

stynkdev@gmail.com | szyrek@stynk.eu | +48 505 784 179 | Tricity, Poland | Remote | Available immediately | LinkedIn | GitHub

ARCHITECTURE LEARNED FROM THE IMPLEMENTATION SIDE

12+ years implementing the transitions architecture is meant to enable - from reconstructing inherited C++/Java EE systems, through integration, selective extraction and platform composition, to distributed systems, production ownership, UX and agentic delivery. I use that accumulated perspective to turn ambiguous business problems into complete systems whose technical complexity disappears behind the right abstraction.

BECOMING BY DOING

DISCOVER · DESIGN · BUILD · OPERATE · EVOLVE

MISYS → FINASTRA 2013-2021 · Gdynia

JUNIOR → EXPERT

●	STAGE 1 Junior → independent engineer		DEBUGGING	LEGACY
			RELIABILITY	
	PROBLEM	Inherited TopOffice and related financial-risk systems arrived with outdated setup notes, unfamiliar repositories, mixed C++/Java EE stacks and little original-team context.		
	INTERVENTION	Built a repeatable investigation workflow around logs, debuggers, stack traces, Git history/bisect, Bash, tests, documentation, source code and finding the right human when needed.		
●	EVIDENCE	Regularly reproduced and fixed failures in systems I had not known existed before receiving the task; grew from local fixes into cross-product ownership.		
	COMPETENCE	Engineering independence is not knowing every technology; it is knowing how to make an unknown system reveal the truth.		
	STAGE 2 Engineer - integration & modernization		INTEGRATION	DISTRIBUTED
			CI/CD	
			IDENTITY	
	PROBLEM	Stabilized legacy products now had to participate in common CI, messaging, SSO and cross-product workflows without rewriting everything that already worked.		
●	INTERVENTION	Added adapters, common contracts, JMS/event flows, OpenID/OIDC integration and deployment standardization; modernized CMR persistence from DataXtend ORM to JPA incrementally while product work continued.		
	EVIDENCE	Helped multiple heterogeneous products move toward common platform contracts while preserving their proven internals and delivery cadence.		
	COMPETENCE	Separate implementation from capability: stable seams, protocols and contracts let very different systems become composable.		
	STAGE 3 Senior - selective extraction & subsystem ownership		SCALING	DISTRIBUTED
			IDENTITY	
●	PROBLEM	Useful legacy capabilities could not scale or integrate cleanly because bottlenecks lived in centralized synchronization, data access or incompatible identity models.		
	INTERVENTION	Extracted scalable/stateless processing around existing calculation engines; built UserManagement plus a shared authentication core in C reused by C++, Java via JNI and Objective-C/GNUstep.		
	EVIDENCE	Owned UserManagement as other teams integrated with it; became a cross-team contact and mentor while the extracted path enabled horizontal scale without rewriting the engine.		
	COMPETENCE	Architecture as constraint reconciliation: change the boundary that blocks the required property, preserve what already works.		
	STAGE 4 Expert - platform perspective		PLATFORM	INTEGRATION
●	PROBLEM	Years of acquisitions had left products, eras and stacks; the business wanted one coherent product instead of exposing each inherited system separately.		
	INTERVENTION	Composed capabilities behind OpenAPI contracts, shared identity/configuration and a Docker Compose integrated environment ; the configurable UI/MVP was MisysBoard , work that evolved into the FusionFabric platform direction.		
	EVIDENCE	Saw CMR/KGR/ALM-era capabilities become a configurable API-driven platform where workflows and dashboards could be assembled above legacy engines.		
	COMPETENCE	A system is an abstraction over capabilities; a platform is a language for composing them into new solutions.		
	STAGE 5 Intel Corporation - Firmware Engineer		FIRMWARE	DEBUGGING
●	PROBLEM	SSD firmware exposed timing, state, memory and hardware constraints normally hidden by managed enterprise runtimes.		
	INTERVENTION	Worked on NAND/storage-driver software in Embedded C and Python test automation , investigating failures across the software/hardware boundary.		
	EVIDENCE	Delivered firmware and automated-verification work inside a specialized SSD development environment.		
	COMPETENCE	Learned firsthand what higher-level abstractions buy - and which hardware/runtime constraints they simplify rather than erase.		
			VERIFICATION	
			DATA	

THE THREAD SO FAR

Legacy debugging taught me to reconstruct systems I did not yet understand; integration moved the focus to stable boundaries; subsystem ownership and platform composition taught me to change the boundary instead of rewriting the world; firmware made runtime constraints physical.

BECOMING BY DOING - CONTINUED

Ciklum / EverC - Full Stack Developer

Nov 2022 - Aug 2023 · Remote

OPERATIONS OBSERVABILITY RELIABILITY LEGACY

PROBLEM

INTERVENTION

EVIDENCE

COMPETENCE

A fast-moving fraud/risk product had to keep shipping while the team also owned deployment and production reliability end to end. Worked mainly on the **Groovy/Grails MerchantView backend**, deployments and on-call; restored failed services under pressure and investigated root causes through Grafana, monitoring and logs; worked with AWS, Terraform and Kubernetes.

Owned software behavior beyond merge: delivery, observability, incident response and follow-up investigation were one engineering loop. Production ownership makes maintainability and observability engineering requirements; velocity lasts only while a system remains understandable and safe to change.

Nordea - Full-stack Developer

Aug 2023 - Mar 2024 · Gdynia

GOVERNANCE SECURITY INTEGRATION CI/CD

PROBLEM

INTERVENTION

EVIDENCE

COMPETENCE

New credit-product features entered a banking ecosystem too large for any one team to control, so work began with discovering data owners, contracts, access paths and non-functional requirements.

Built Java/Angular microservice and microfrontend flows in team-owned components: teams chose internals, while precise external contracts and security, performance, testing and integration gates protected the ecosystem.

Delivered flows where independently owned components could evolve internally yet remain composable through stable contracts and standardized validation.

At organizational scale, coherence moves from shared implementation rules to ownership boundaries, stable contracts and independently verifiable behavior.

Hapag-Lloyd AG - Full Stack Engineer

Jun 2024 - Mar 2025 · Gdańsk

INTEGRATION DATA PLATFORM UX

PROBLEM

INTERVENTION

EVIDENCE

COMPETENCE

Global logistics reconciled events and state across the transport chain inside a mature, strongly opinionated internal DDD platform.

Worked on real-time container-tracking/event flows and maintenance through higher-level framework abstractions for domain models, validation and UI rather than rebuilding plumbing per feature.

Contributed inside a mature platform that hid much of the technical machinery behind domain-oriented building blocks and operational contracts.

The key platform boundary is between the user's/domain model and the machinery that makes the abstraction true - including ownership and failure handling.

Bridge · software delivery itself becomes an architecture problem

Apr-Sep 2025 · independent R&D

Used coding agents as an **execution multiplier** and experimented with documentation-driven context, reviewer agents, evidence gates, triggerable roles and local models. The lesson: context, responsibility boundaries, feedback, validation and escalation determine whether autonomy stays reliable.

Stynk - Independent Systems & Product Engineer / Technical Owner

Oct 2025 - Present · Poland

REQUIREMENTS BUSINESS UX PLATFORM OPERATIONS SECURITY AI/AGENTS

PROBLEM

INTERVENTION

EVIDENCE

COMPETENCE

A growing construction company could not scale because the owner was the human message bus: calls, paper, notes, email and memory held the shared state. The first request was simply **capture data from a paper contract**.

Did end-to-end discovery, modeled domain/workflows, designed UX and security, built and operate the CRM/site/infrastructure, moved coordination into auditable notifications/escalations, and generalized changing pricing/service logic into typed dynamic Studio models and rules.

~20-25 users handle ~120-150 contracts/year in one searchable system; repository ~130k LOC with ~1,600 tests. Delivery runs through staging/CI/E2E/preflight/rollback automation; since launch, no data-loss incident or user-blocking outage.

Integrated the earlier lessons into one practice: discover the bottleneck, choose the right abstraction, expose it in the user's language, automate guarantees underneath, and keep changing business models owned by the business.

Stynk case study ↗

visual tour ↗

Current R&D · HackaTeam + VibeGuard

extending the same ownership question into agentic engineering

Execution side - HackaTeam. GitHub-native, documentation-first agent loops with CI/evidence feedback, local/hosted execution and state-driven work selection. The point is bounded, observable delegated engineering - not merely "LLM writes code".

Ownership side - VibeGuard. Tech Owner control-loop: routine evidence and policy below the boundary; explicit human authority for architecture, security, irreversible semantics, exceptions and decision memory above it.

AGENTS AUTOMATION PLATFORM EVIDENCE

HackaTeam ↗ VibeGuard ↗

WHAT THAT ADDS UP TO

I work best where business space meets implementation space. I can discover an ambiguous operational problem, model the domain and ownership boundaries, design the architecture and UX that expose the right capability, and own the result through production. I did not learn that as a catalogue of patterns; I learned it by implementing the transitions - and Stynk is the first place I have consciously applied the whole chain end to end.